

TREES

Trees: Basic Terminology in Trees.

Binary Trees: - Properties, Representation of Binary Trees using Arrays and Linked lists.

Binary Search Trees:- Basic Concepts, BST Operations: Insertion, Deletion, Tree Traversals.

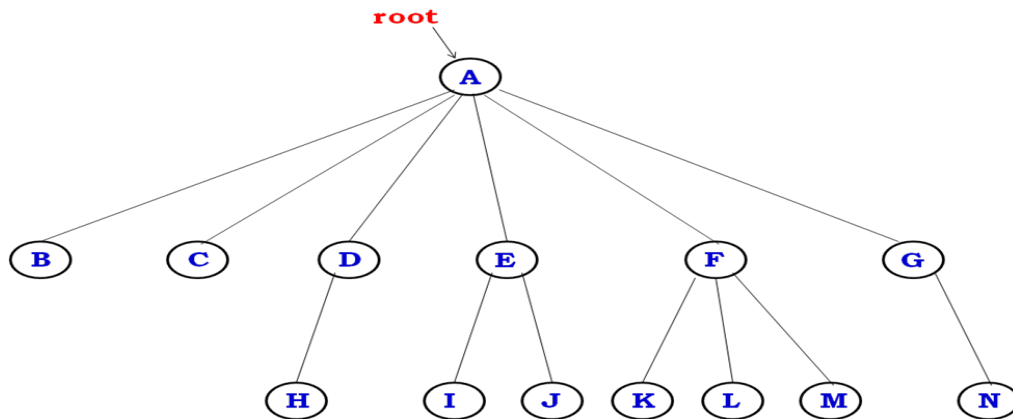
Applications:- Expression Trees, Heap Sort.

Balanced Binary Trees:- AVL Trees, Insertion, Deletion and Rotations.

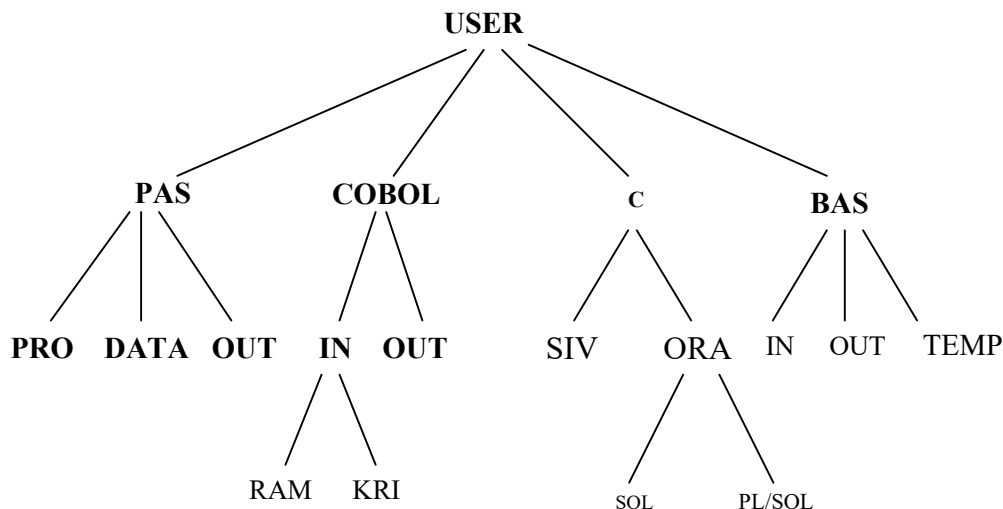
TREES

A Tree is a finite set of one or more nodes such that

1. There is a specially designated node called the Root.
2. The remaining nodes are partitioned into $n \geq 0$ disjoint sets $T_1 T_2 T_3 \dots T_n$.
Where each of these sets is a Tree.

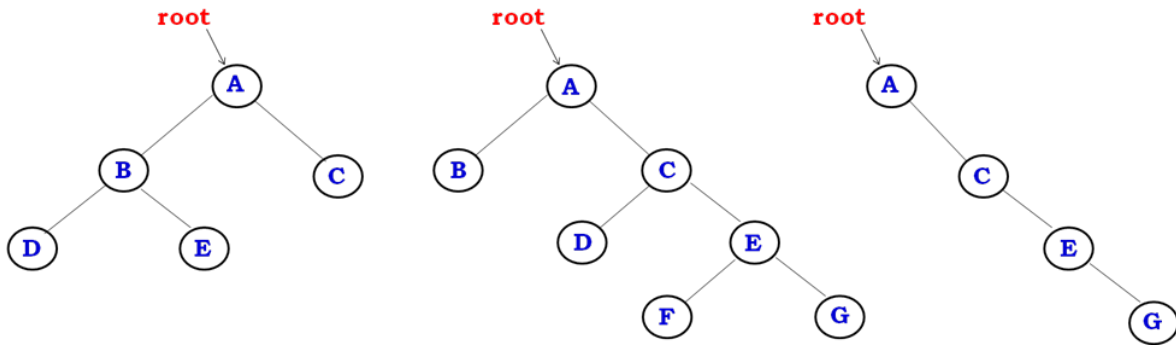


A node stands for the item of information plus the branches to other items. The number of subtrees of a node is called its **degree**. Nodes that have degree ZERO are called **Leaf** nodes or Terminal nodes. There are many applications for trees . One of the popular uses is the *directory structure* in many common operating systems including UNIX, DOS and Windows.



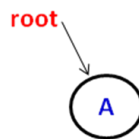
Binary Trees

Definition: Binary tree is either empty, or it consists of a node called the root together with two binary trees called the left sub-tree and the right sub-tree.



A Binary tree is a tree in which no node can have more than two children.

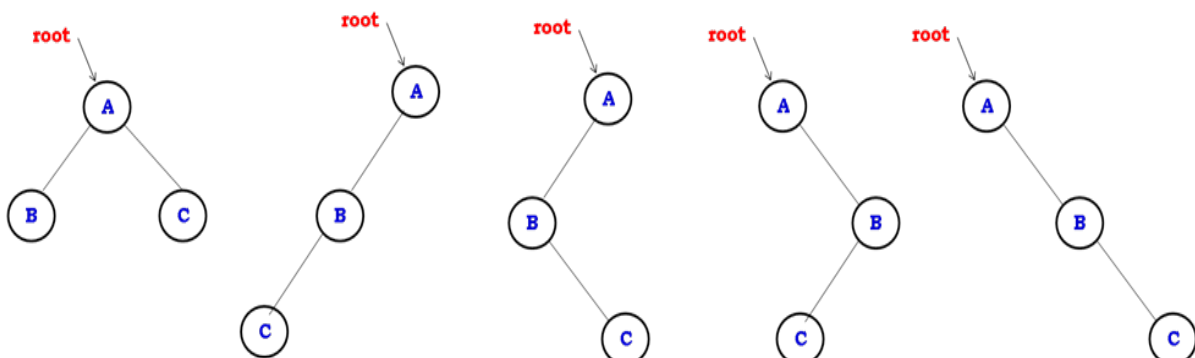
To construct a binary tree with one node is to make that node its root and to make both left and right sub trees empty.



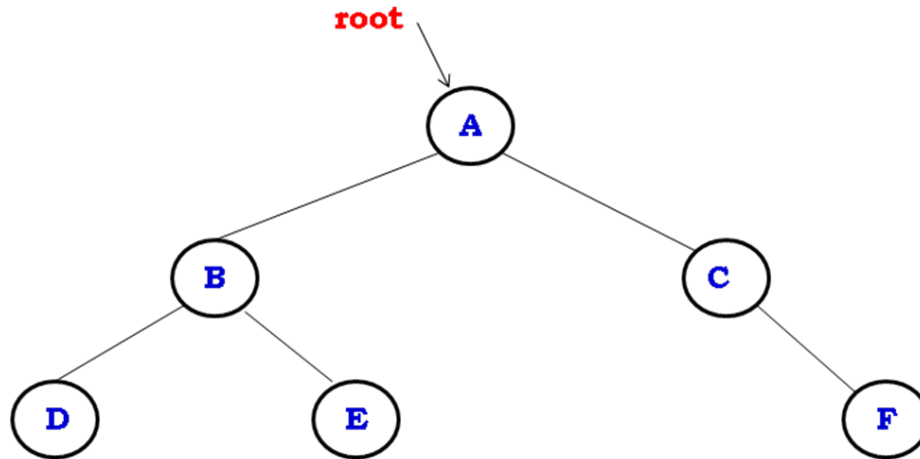
With two nodes in the tree, one of them will be the root and the other will be in a sub tree. Thus either the left or the right sub tree must be empty, and the other will contain exactly one node.



Binary tree with three nodes.

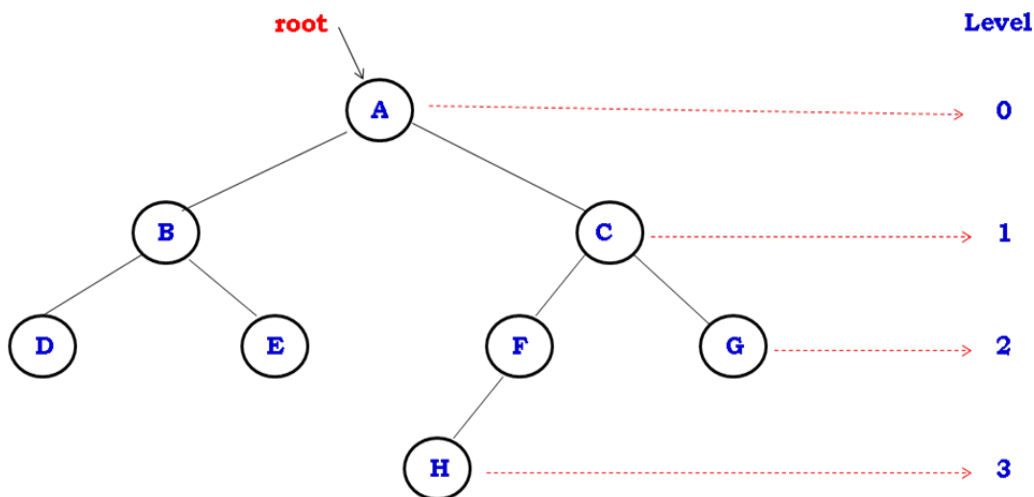


Consider the following Binary Tree.



Each element of a binary tree is called Node of the tree. Node 'A' is the root of the tree and node 'B' is the root of its left subtree. Node 'A' is said to be the father of node 'B' and node 'B' is said to be the left son of node 'A'. A node that has no sons is called a Leaf node. D, E and F are leaf nodes of the above binary tree.

Level: The level of a node in a binary tree is defined as follows: The root of the tree has level 0 and the level of any other node in the tree is one more than the level of the father.

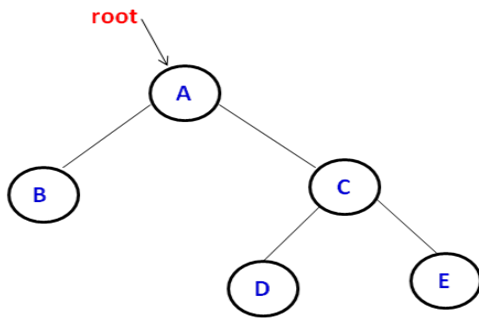


Height / Depth: The height or depth of a binary tree is the maximum level of any leaf in the tree. This equals the length of the longest path from the root to any leaf. The depth of the above tree is 3.

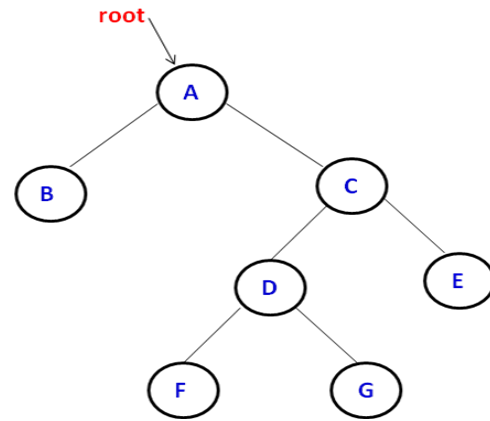
Strictly Binary Tree/ Full Binary Tree

If every non leaf node in a binary tree has non empty left and right sub-trees, then the tree is termed as a strictly binary tree or full binary tree.

A strictly binary tree or full binary tree with 'n' leaves always contains $2 * n - 1$ nodes.



$$\begin{aligned}\text{No of leaf nodes} &= 3 \\ \text{Total nodes} &= 2*3 - 1 \\ &= 6-1 \\ &= 5\end{aligned}$$



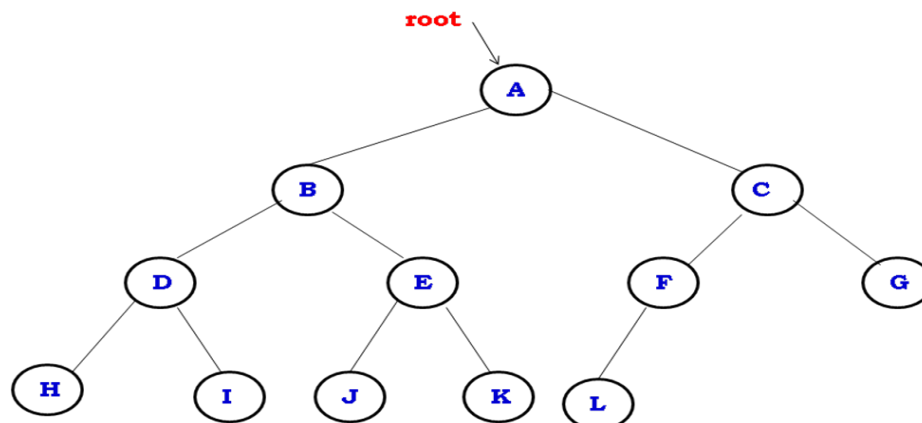
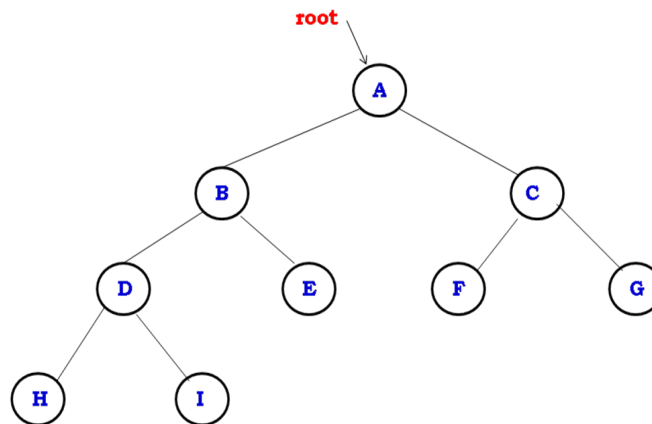
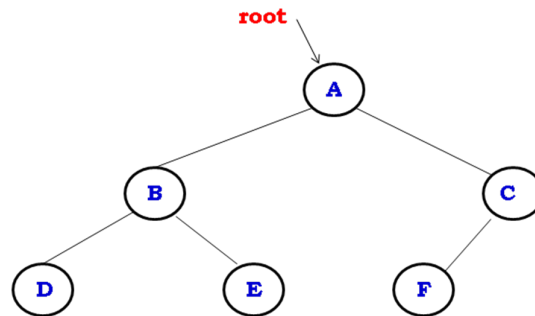
$$\begin{aligned}\text{No of leaf nodes} &= 4 \\ \text{Total nodes} &= 2*4 - 1 \\ &= 8 - 1 \\ &= 7\end{aligned}$$

A Binary Tree is a full binary tree or a strictly binary tree if every node has 0 or 2 children. The following are the examples of a full binary tree. We can also say a full binary tree is a binary tree in which all nodes except leaf nodes have two children.

Complete Binary Tree/ Binary Heap

A Binary Tree is a Complete Binary Tree if all the levels are completely filled except possibly the last level and the last level has all keys as left as possible.

Practical example of Complete Binary Tree is Binary Heap.



Perfect Binary Tree

A Binary tree is a Perfect Binary Tree in which all the internal nodes have two children and all leaf nodes are at the same level.

We can clearly say that any level 'L' of a binary tree 'T', there should be at most 2^L nodes.

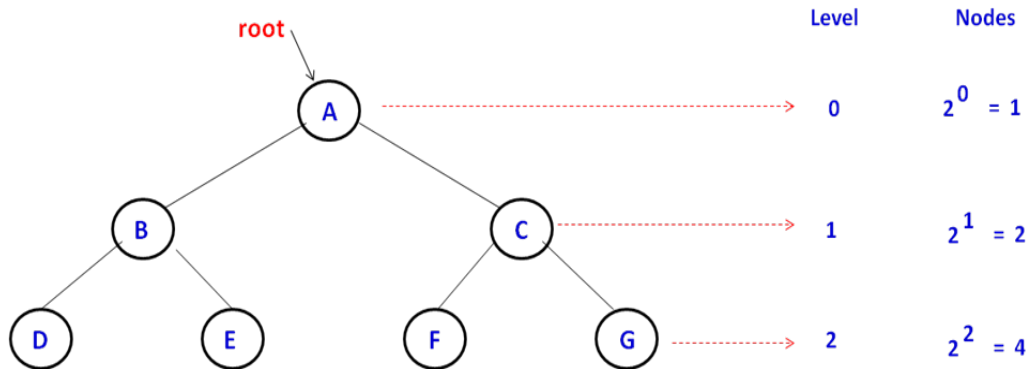


Figure 1) Perfect Binary Tree of depth 2

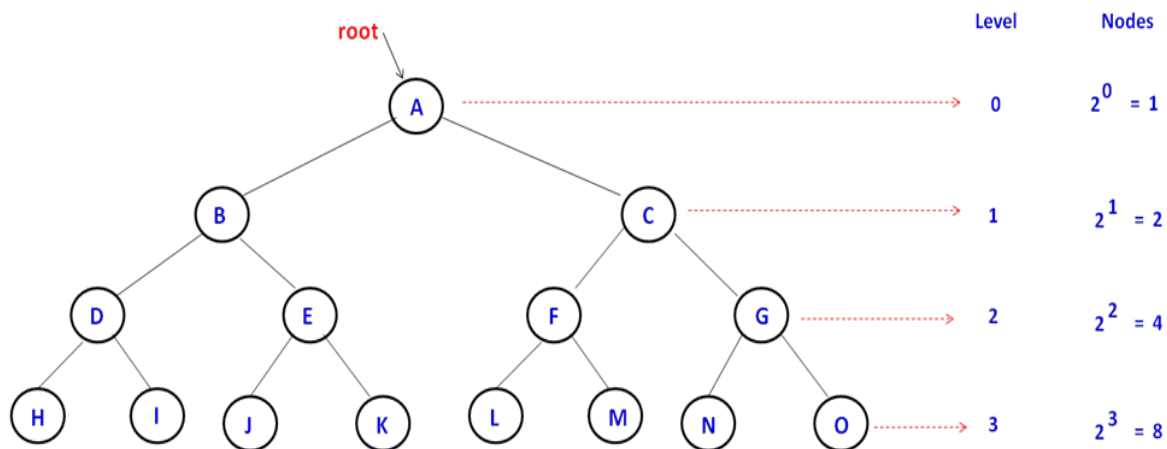


Figure 2) A perfect Binary Tree of depth 3

A perfect binary tree of depth d is the binary tree of depth d that contains exactly 2^L nodes at each level 'L' between 0 and d.

Level 0 contains	2^0 nodes	i.e.	1 node
Level 1 contains	2^1 nodes	i.e.	2 nodes
Level 2 contains	2^2 nodes	i.e.	4 nodes
Level 3 contains	2^3 nodes	i.e.	8 nodes

Total no of nodes in a perfect binary tree of depth D is the sum of the number of nodes at each level between 0 to D i.e.,

$$2^0 + 2^1 + 2^2 + \dots + 2^D$$

The above perfect binary tree of depth 3 contains 15 nodes, which is shown as follows.

$$\begin{array}{r} 2^0 + 2^1 + 2^2 + 2^3 \\ 1 + 2 + 4 + 8 \\ 15 \end{array}$$

by induction

$$2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^d = 2^{d+1} - 1$$

$$\text{Total no of leaves} = 2^3 = 2^3 = 8$$

$$\text{Total no of non-leaves} = 2^d - 1 = 2^3 - 1 = 8 - 1 = 7$$

$$\text{Total no of nodes} = 2^d + 2^d - 1$$

$$= 2 * 2^d - 1$$

$$= 2^{d+1} - 1$$

$$= 2^{3+1} - 1$$

$$= 2^4 - 1$$

$$= 16 - 1$$

$$= 15$$

Binary Tree Path

What is a path in a binary tree? A path is a collection of nodes from the root to any leaf of the tree. By definition, a leaf node is a node which does not have left or right child.

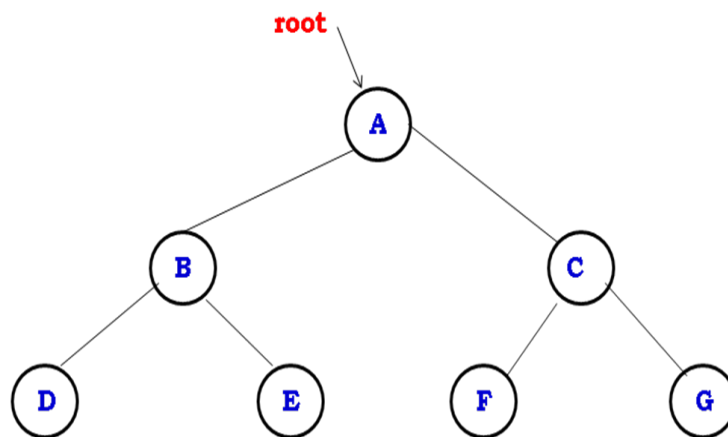
For example, the following binary tree is having 4 leaf nodes. There are 4 paths in this binary tree i.e., paths from root to leaf nodes. The following are paths of the binary tree.

A, B, D

A, B, E

A, C, F

A, C, G



Implementation of a Binary Tree Node

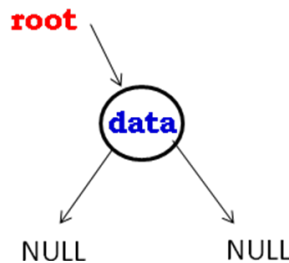
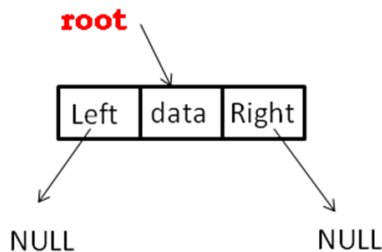
The declaration of tree nodes is similar in structure to that for doubly linked lists, in that a node is a structure consisting of the key information plus two pointers (left and right) to other nodes.

Binary trees have many important uses not associated with searching. One of the principal use of binary trees is in the area of compiler design.

```
struct node
{
    int data;
    struct node *left, *right;
};
typedef struct node treenode;
```

```
treenode *root;
treenode *p;
```

```
p =(treenode *)malloc(sizeof(treenode));
p->data=x;
p->left=NULL;
p->right=NULL;
```



Traversals of Binary Tree

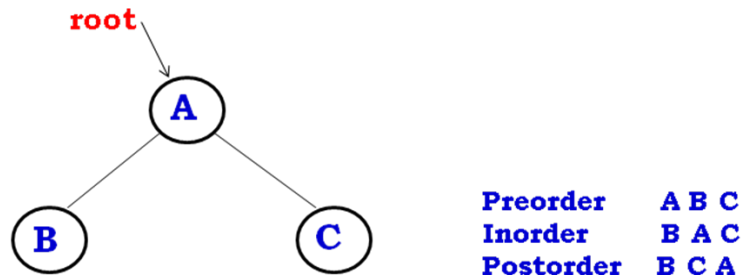
One of the most important operations on a binary tree is traversal, moving through all the nodes of the binary tree, visiting each one in turn.

We can traverse the binary tree in many ways.

Preorder	VLR
Inorder	LVR
Postorder	LRV

With preorder traversal, the node is visited before the subtrees.
With inorder traversal, the node is visited between the subtrees.
With postorder traversal, the node is visited after the subtrees.
In all cases left is visited before right.

Consider the following binary tree.



To traverse a non-empty binary tree in preorder, we perform the following three operations.

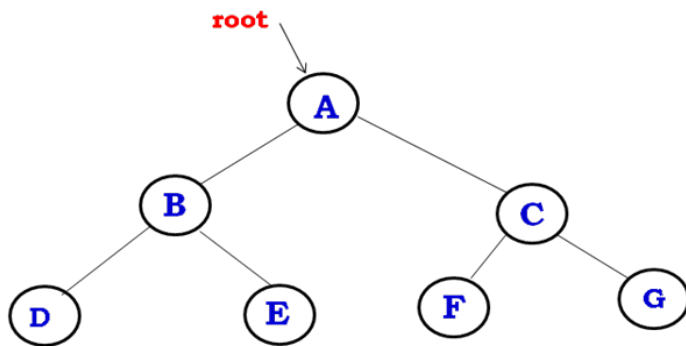
- (i) Visit the root
- (ii) Traverse the left subtree in preorder
- (iii) Traverse the right subtree in preorder

To traverse a non-empty binary tree in inorder, we perform the following three operations.

- (i) Traverse the left subtree in inorder.
- (ii) Visit the root.
- (iii) Traverse the right subtree in inorder.

To traverse a non-empty binary tree in postorder, we perform the following three operations.

- (i) Traverse the left subtree in postorder.
- (ii) Traverse the right subtree in postorder.
- (iii) Visit the root.



Preorder A B D E C F G
Inorder D B E A F C G
Postorder D E B F G C A

```

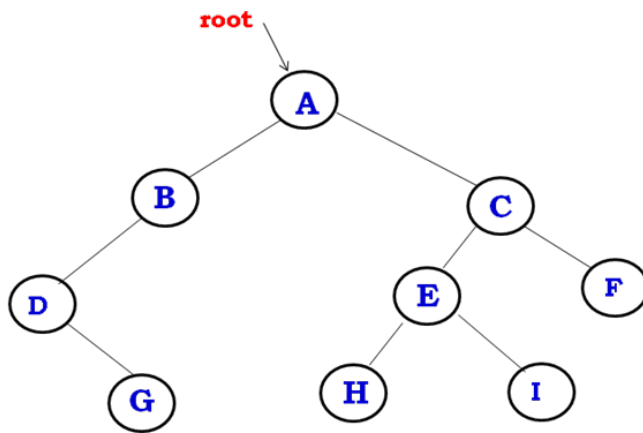
/* -----Function to traverse a binary tree in preorder */
preorder(p)
treenode *p
{
    if (p != NULL)
    {
        printf("%d", p->data);
        preorder(p->left);
        preorder(p->right);
    }
}
  
```

```

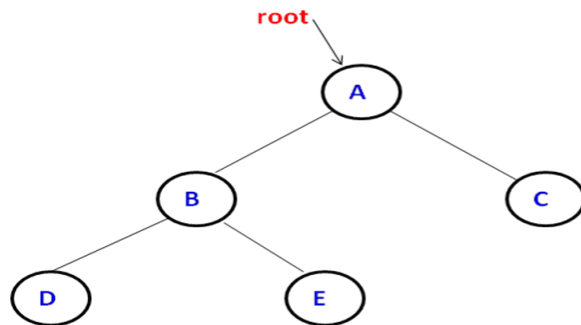
/*----- Function to traverse a binary tree in inorder */
inorder(p)
treenode *p;
{
    if (p != NULL)
    {
        inorder(p->left);
        printf("%d", p->data);
        inorder(p->right);
    }
}
  
```

```

/*----- Function to traverse a binary tree in postorder */
postorder(p)
treenode *p;
{
    if (p != NULL)
    {
        postorder(p->left);
        postorder(p->right);
        printf("%d", p->data);
    }
}
  
```



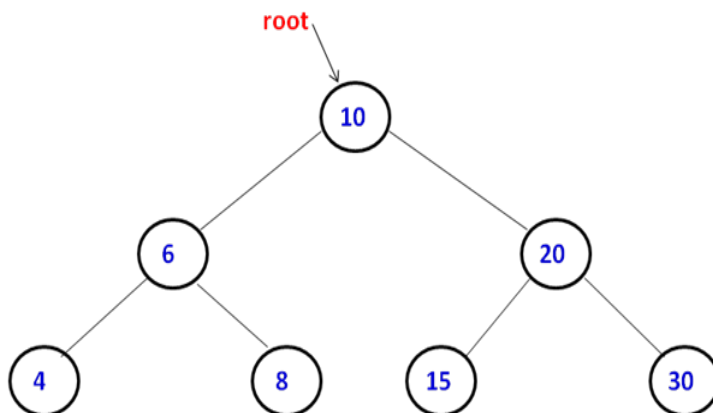
Preorder A B D G C E H I F
 Inorder D G B A H E I C F
 Postorder G D B H I E F C A



Preorder A B D E C
 Inorder D B E A C
 Postorder D E B C A

Preorder(A) = V	L	R
= visit A	preorder(B)	preorder(C)
Preorder(B) = visit B	preorder(D)	preorder(E)
Preorder(D) = visit D	preorder(NULL)	preorder(NULL)
	Backtracking	
Preorder(E) = visit E	preorder(NULL)	preorder(NULL)
	Backtracking	
Preorder(C) = visit C	preorder(NULL)	preorder(NULL)

So the order of the nodes visited in preorder are A B D E C



Preorder 10 6 4 8 20 15 30
 Inorder 4 6 8 10 15 20 30
 Postorder 4 8 6 15 30 20 10

Expression Trees

Consider the expression $(A + B * C) + ((D * E + F) * G)$

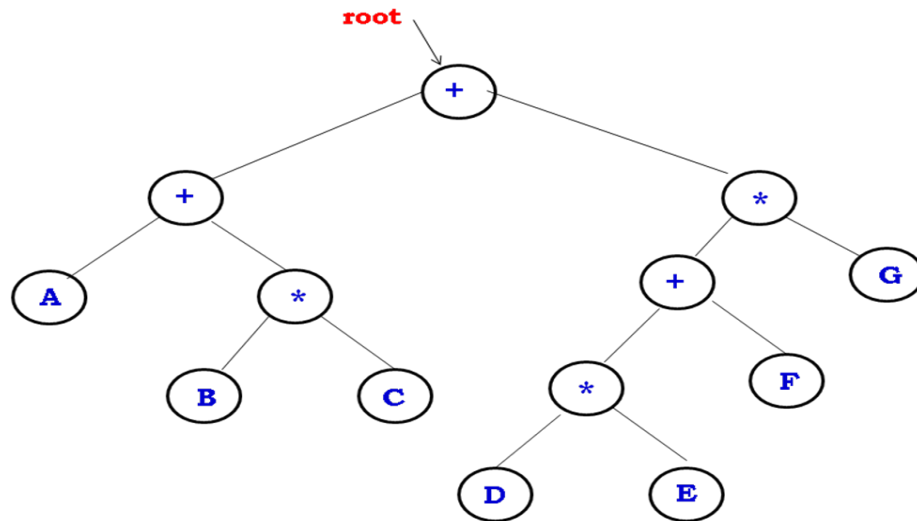
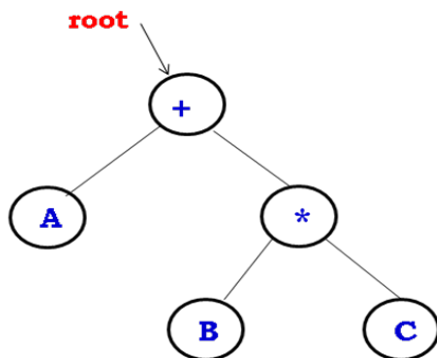


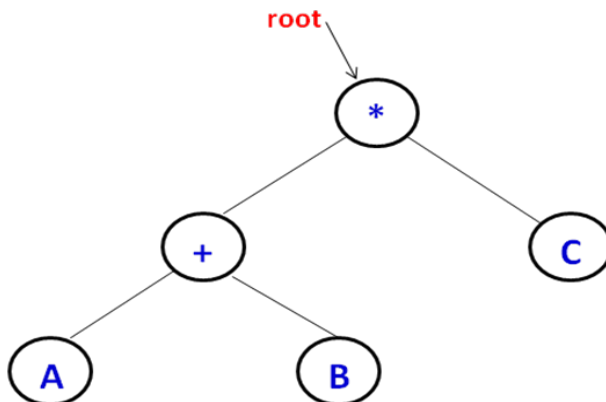
Figure shows an example of an expression tree. The leaves of an expression tree are operands and non-leaf nodes are operators.

Example 1) $A + B * C$



Preorder $+ A * B C$
Inorder $A + B * C$
Postorder $A B C * +$

Example 2) $(A + B) * C$



Preorder $* + A B C$
Inorder $A + B * C$
Postorder $A B + C *$

Binary Search Tree

Definition: A Binary Search Tree is a binary tree that is either empty or in which every node contains a key and satisfies the following conditions.

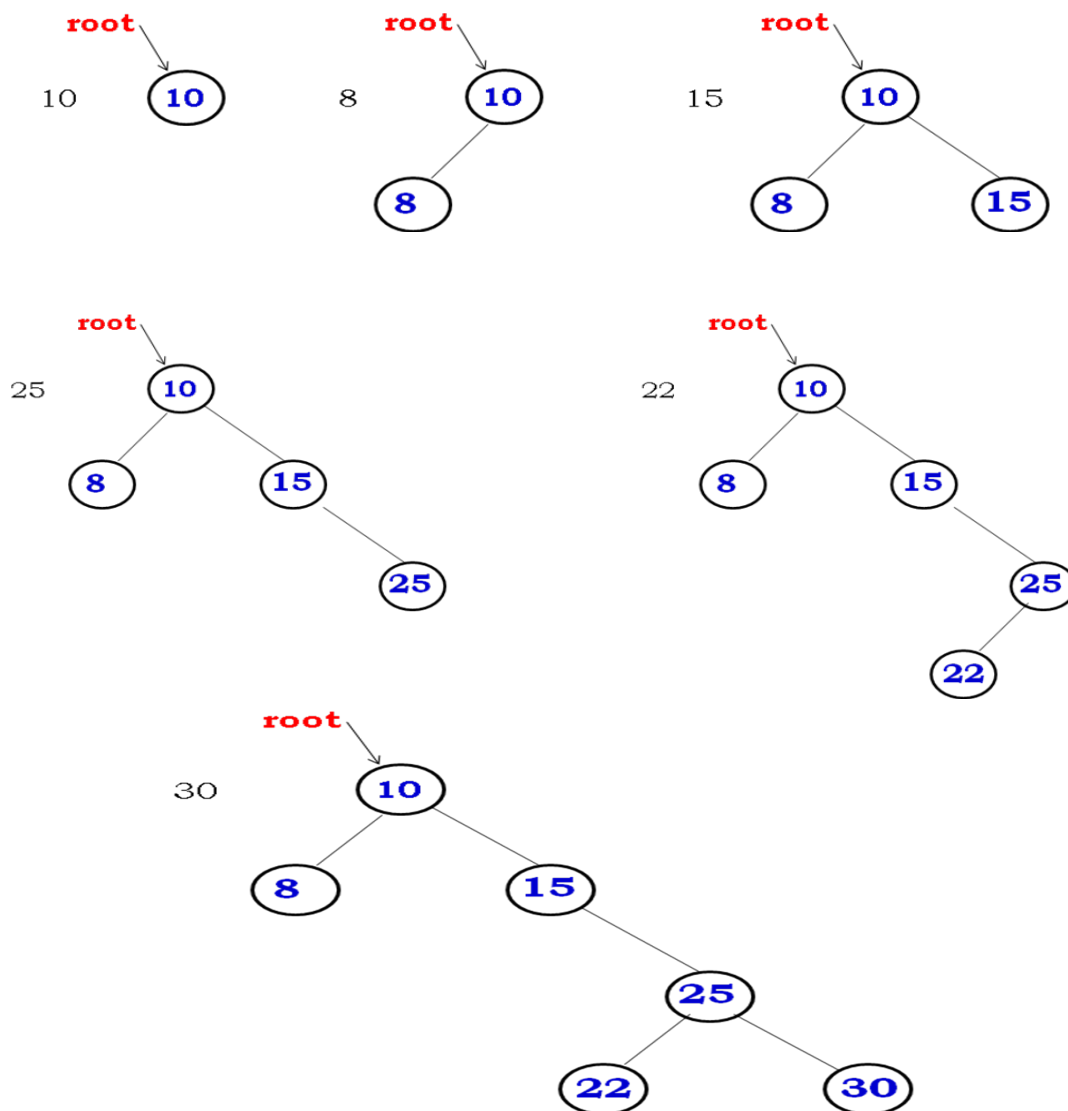
1. The key in the left child of a node (if it exists) is less than the key in its parent node.
2. The key in the right child of a node is greater than the key in its parent node.
3. The left and right sub trees of the root are again binary search trees.

No two entries in a binary search tree may have equal keys.

Example 1) Creating a binary search tree with the following values.

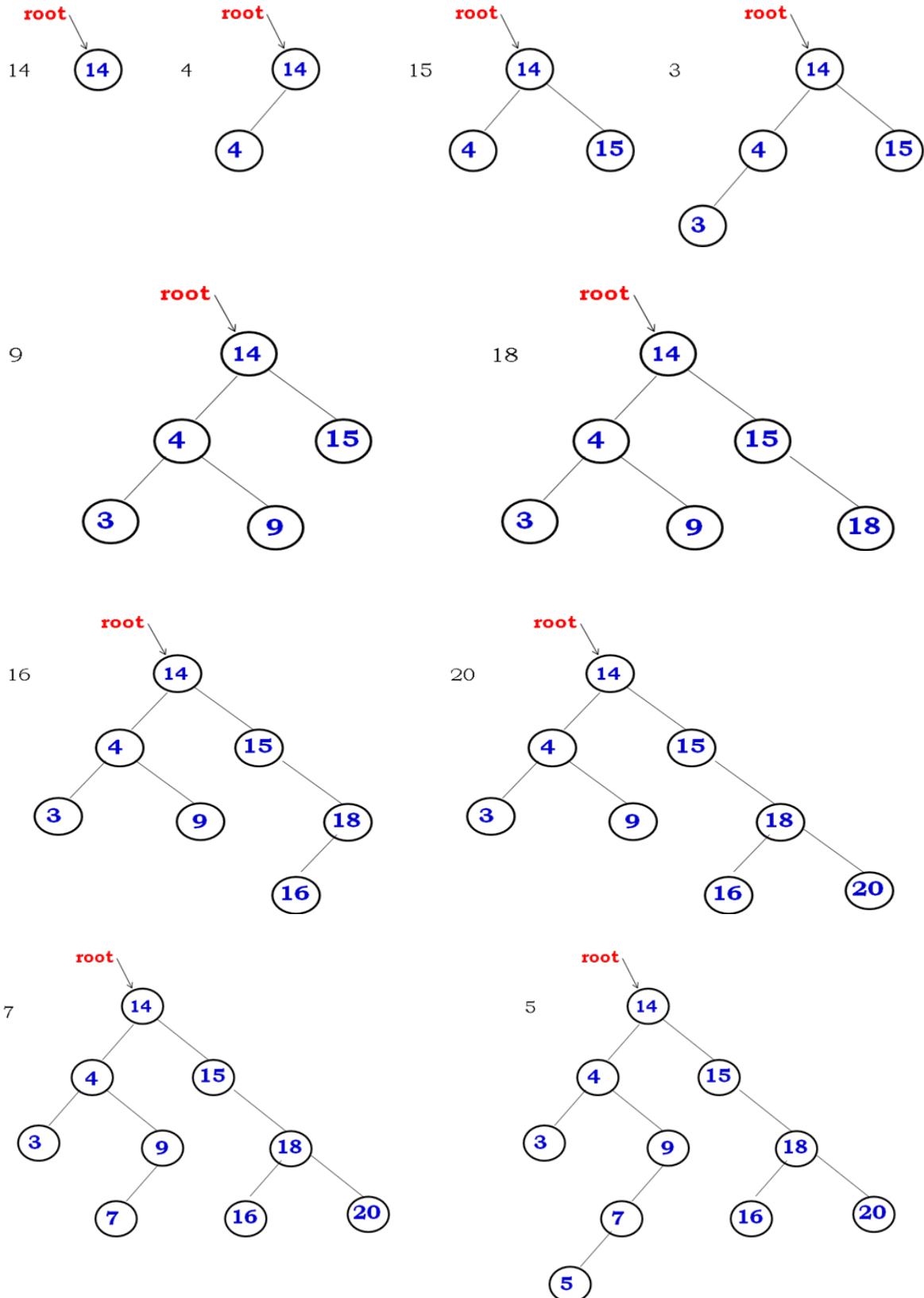
10 8 15 25 22 30

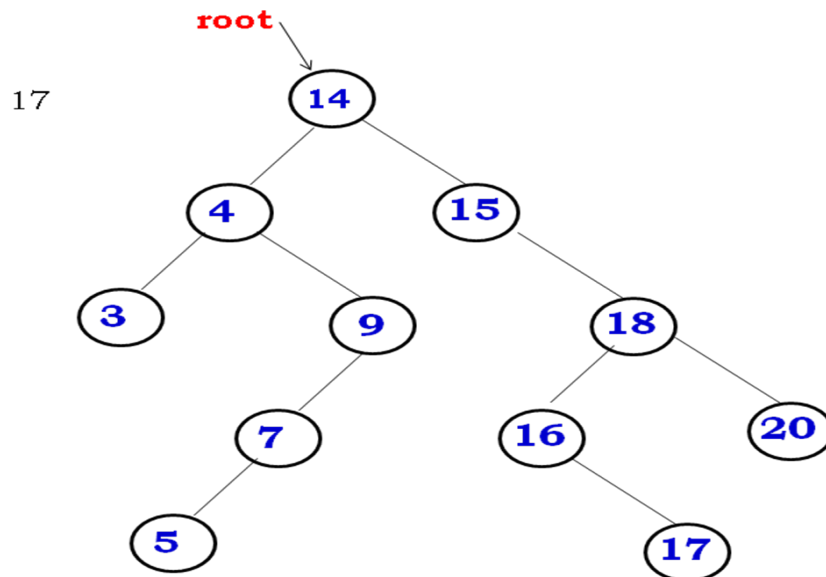
Step by step creation of a binary search tree.



Example 2) Creating a binary search tree with the following values.

14 4 15 3 9 18 16 20 7 5 17





Because of the recursive definition of trees, it is common to write these routines recursively. The average depth of a binary search tree is $O(\log n)$.

Creating a Binary Search Tree:

The `buildtree()` routine is conceptually simple. To insert X into tree T proceed down the tree right from root node. If X is found, do nothing. Otherwise insert X at the last spot on the path traversed.

Each time we compare X with the value and if it is greater than the node value, we check if the right node is NULL or not. If it is NULL we add this node as right node of the current node. If the right node is not NULL, we go to that node and again we compare X with the value of that node.

If X is less than the node value, we check if the left node is NULL or not. If it is NULL we add this node as left node of the current node. If the left node is not NULL, we go to that node and again we compare X with the value of that node. Repeat this process until you add the node with value X to the binary search tree.

```

void buildtree(p,x)
treenode *p;
int x;
{
    treenode *q;

    if ( x > p->data )
        if (p->right == NULL)
        {
            q=(treenode *)malloc(sizeof(treenode));
            q->data=x;
            q->left=NULL;
            q->right=NULL;
            p->right=q;
        }
        else
            buildtree(p->right,x);
    else
        if ( x < p->data )
            if (p->left == NULL)
            {
                q=(treenode *)malloc(sizeof(treenode));
                q->data=x;
                q->left=NULL;
                q->right=NULL;
                p->left=q;
            }
            else
                buildtree(p->left,x);
        else
            printf("Duplicate node \n");
    }
}

```

Finding the number of leafs in a Binary Search Tree

We know how to traverse a binary search tree. Traversing is nothing but visiting each and every node of the Binary search tree. We have 3 traversal techniques. In all the 3 traversal techniques we visit all the nodes of the binary search tree.

Now while visiting the nodes check whether the node is a leaf or not. A node is called leaf node if the left and right links of the node are pointing to NULL.

You can take any traversal technique. Instead of visiting the node compare whether the left and right links are NULL or not. If the left and right links are NULL then you can increase the count of leaf nodes. By the end of the traversal you can get the count of leaf nodes.

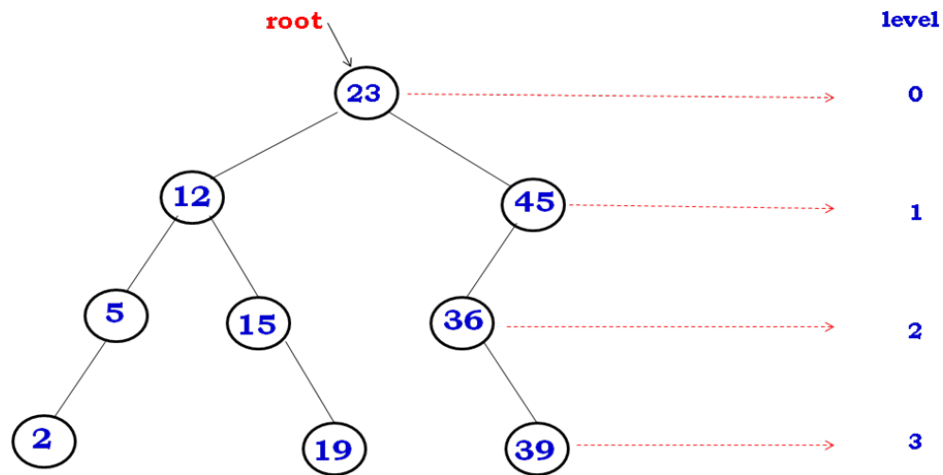
```
/*----- Counting LEAF nodes */
void leafcount(p,cnt)
treenode *p;
int *cnt;
{
    if(p!=NULL)
    {
        leafcount(p->left,cnt);
        leafcount(p->right,cnt);
        if((p->left==NULL) && (p->right==NULL))
            (*cnt)++;
    }
}
```

In the above function we have taken the postfix type of traversal. To count the number of leaf nodes we have used a variable cnt and assign 0. Instead of printf statement we are comparing the left and right links for NULL. If both pointing to NULL count will be increased.

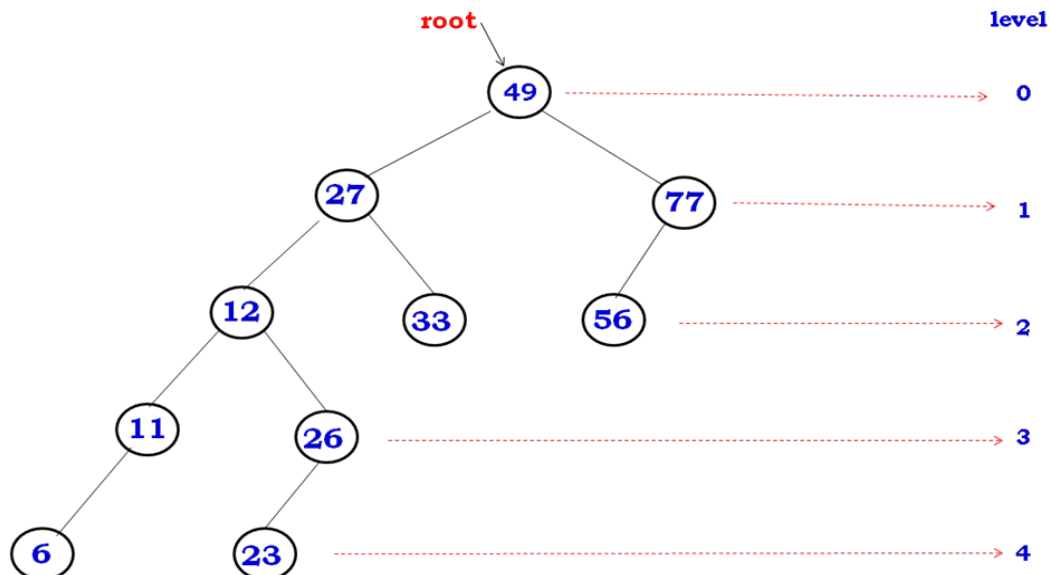
Finding Height / Depth of a binary search tree

The height of the binary tree is the longest path from root node to any leaf node in the tree.

Every binary tree has a root. root may have two subtrees. Find height of sub trees . Height of binary tree is defined as $1 + \max$ height of left and right sub trees.



A Binary Search tree of height /depth 3



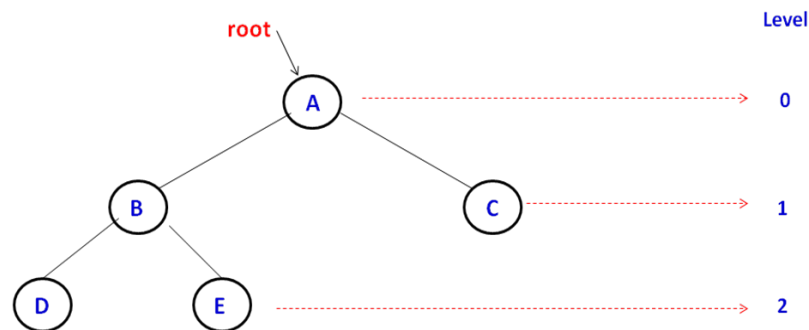
A Binary Search Tree of Height 4

```

/*----- to find largest of two heights */
int max(a,b)
int a,b;
{
    if (a>b)
        return(a);
    else
        return(b);
}

/*-----to find height of BST*/
int height(p)
treenode *p;
{
    if (p==NULL)
        return(-1);
    else
        return(1+max(height(p->left),height(p->right)));
}

```



$\text{Height}(A) = 1 + \max(\text{height}(B), \text{height}(C))$
 $\text{Height}(B) = 1 + \max(\text{height}(D), \text{height}(E))$
 $\text{Height}(D) = 1 + \max(\text{height}(\text{NULL}), \text{height}(\text{NULL}))$
 $= 1 + \max(-1, -1)$
 $= 1 + (-1) = 0$

$\text{Height}(E) = 1 + \max(\text{height}(\text{NULL}), \text{height}(\text{NULL}))$
 $= 1 - 1 = 0$

height(D) and height(E) are substituted in height(B) expr.

$\text{Height}(B) = 1 + \max(0, 0)$
 $= 1 + 0$
 $= 1$

$\text{Height}(C) = 1 + \max(\text{height}(\text{NULL}), \text{height}(\text{NULL}))$
 $= 1 + \max(-1, -1)$
 $= 1 + (-1)$
 $= 0$

Now substitute height(B) and height(C) in height(A) expr.

$\text{Height}(A) = 1 + \max(1, 0)$
 $= 1 + 1$
 $= 2$

Hence the height of the tree is 2.

Problem 1: The preorder and inorder traversals of a binary tree are

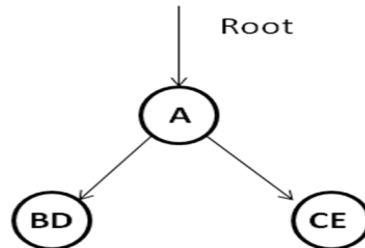
Preorder – A B D C E

Inorder – D B A C E

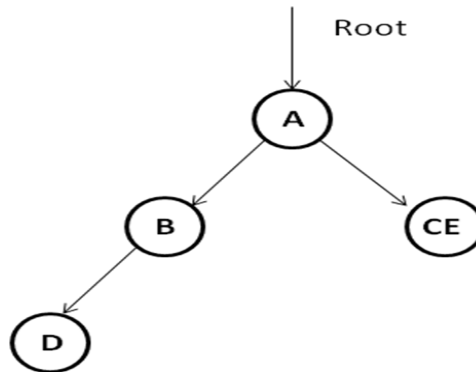
Construct a binary tree and Give the postorder traversal for the above tree.

Solution:

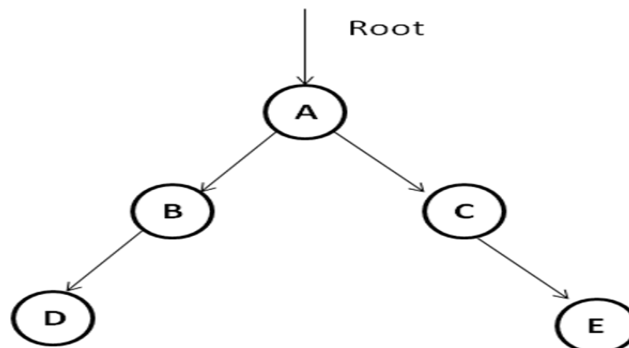
The first letter in preorder must be the root. So '**A**' is our root. By the definition of inorder, all nodes preceeding '**A**' must occur in the left subtree and all nodes succeeding '**A**' must occur in right subtree.



At this stage by seeing the preorder '**B**' comes before '**D**'. So '**B**' is the next root and from the inorder we see '**B**' has an empty right subtree and '**D**' is in it's left subtree.



At this stage by seeing the preorder '**C**' comes before '**E**'. So '**C**' is the next root and from the inorder we see '**C**' has an empty left subtree and '**E**' is in its right subtree.



The Post Order Traversal : DBECA

Problem 2: The preorder and inorder traversals of a binary tree are

Preorder – A B C D E F G H I

Inorder – B C A E D G H F I

Construct the binary tree and Give the postorder traversal for the above tree.

Solution:

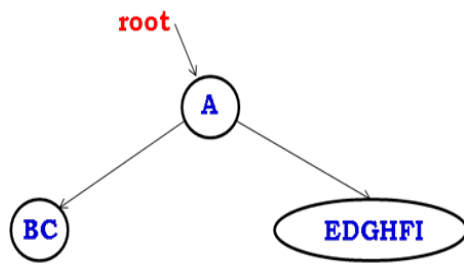


Figure 1

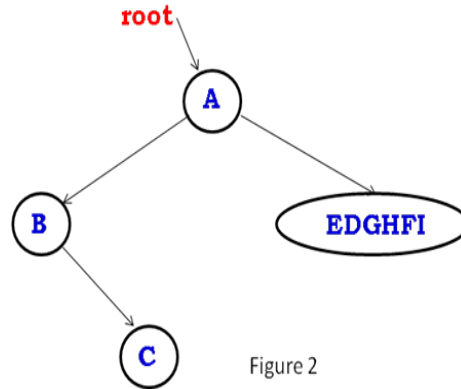


Figure 2

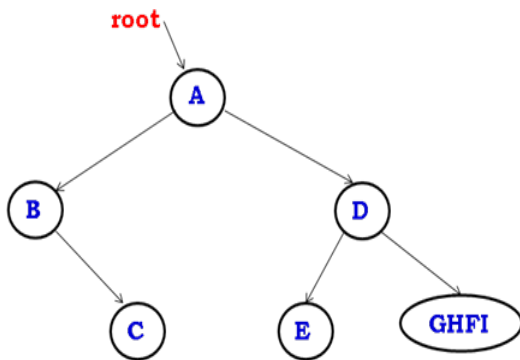


Figure 3

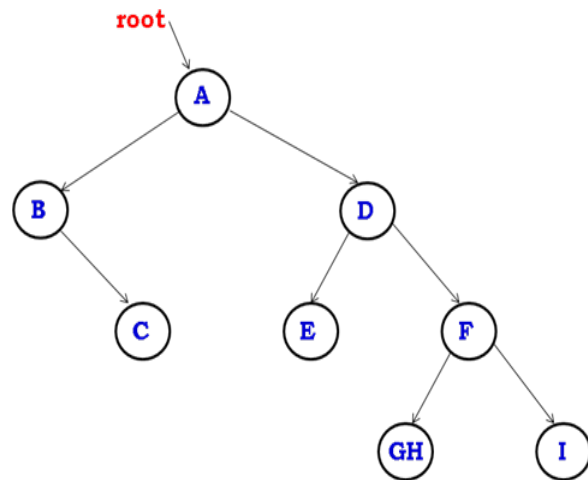


Figure 4

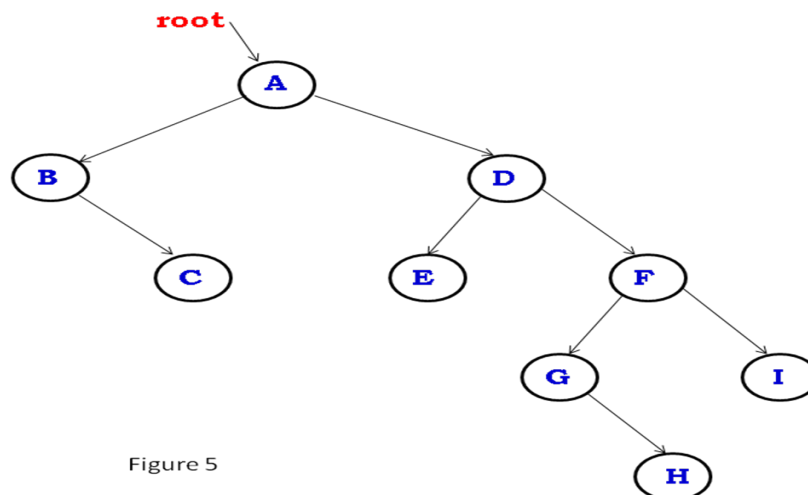


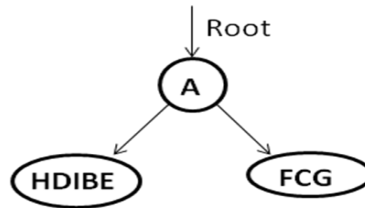
Figure 5

The Post Order Traversal: C B E H G I F D A

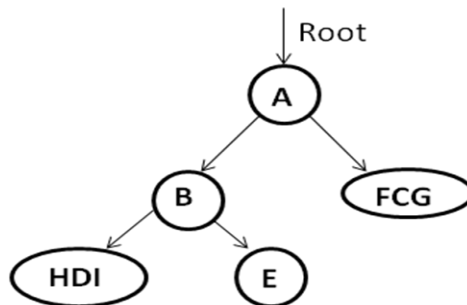
**Problem 3) Build a tree using Postorder : HIDEBFGCA
Inorder : HDIBEAFCG**

Solution:

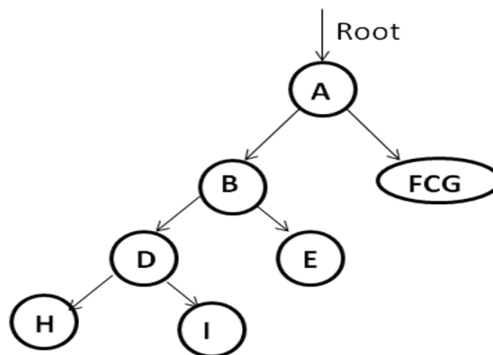
Step-1) The last node in the post order is root node. Now from inorder left sequence to 'A' indicates left subtree and right sequence to 'A' indicates right subtree.



Step 2) If we observe the left sub tree HDIBE from the postorder and inorder
Postorder : H I D E (B)
Inorder : H D I (B) E

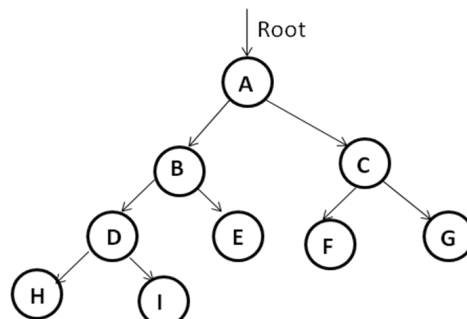


Step 3) If we observe the left sub tree HDI from the postorder and inorder
Post order : H I (D)
Inorder : H (D) I



Step 4) If we observe the right sub tree of root node FCG from the postorder and inorder.

Post order : F G (C)
Inorder : F (C) G



Preorder Traversal : ABDHIECFG

```

/*-----
Program to create a Binary Search Tree and
To traverse the Binary Search Tree in inorder, preorder and postorder
To count the leaf nodes
To find the height of BST
-----*/

#include <stdio.h>
#include <conio.h>
#include <malloc.h>
struct node
{
    int data;
    struct node *left;
    struct node *right;
};
typedef struct node treenode;

/*----- Function to create a binary tree -----*/
/*----- Adds a node to a Binary tree -----*/
void buildtree(p,x)
    treenode *p;
    int x;
{
    treenode *q;

    if ( x > p->data )
        if (p->right == NULL)
        {
            q=(treenode *)malloc(sizeof(treenode));
            q->data=x;
            q->left=NULL;
            q->right=NULL;
            p->right=q;
        }
        else
            buildtree(p->right,x);
    else

        if ( x < p->data )
            if (p->left == NULL)
            {
                q=(treenode *)malloc(sizeof(treenode));
                q->data=x;
                q->left=NULL;
                q->right=NULL;
                p->left=q;
            }
            else
                buildtree(p->left,x);
        else
            printf("Duplicate node \n");
    }
}

```

```

/*----- Preorder Traversal -----*/
void preorder(p)
treenode *p;
{
    if(p!=NULL)
    {
        printf("%d ",p->data);
        preorder(p->left);
        preorder(p->right);
    }
}

/*----- Inorder Traversal -----*/
void inorder(p)
treenode *p;
{
    if(p!=NULL)
    {
        inorder(p->left);
        printf("%d ",p->data);
        inorder(p->right);
    }
}

/*----- Postorder Traversal -----*/
void postorder(p)
treenode *p;
{
    if(p!=NULL)
    {
        postorder(p->left);
        postorder(p->right);
        printf("%d ",p->data);
    }
}

/*----- Counting LEAF nodes -----*/
void leafcount(p,cnt)
treenode *p;
int *cnt;
{
    if(p!=NULL)
    {
        leafcount(p->left,cnt);
        leafcount(p->right,cnt);
        if((p->left==NULL)&&(p->right==NULL))
            (*cnt)++;
    }
}

```

```

/*-----function to find large-----*/
int max(a,b)
int a,b;
{
    if (a>b)
        return(a);
    else
        return(b);
}

/*-----To find height of BST-----*/
int height(p)
treenode *p;
{
    if(p==NULL)
        return(-1);
    else
        return(1+max(height(p->left),height(p->right)));
}

/*-----main function-----*/
void main()
{
    treenode *root;
    char opt;
    int x,cnt,ht;

    clrscr();

    printf ("enter the root value ");
    scanf("%d",&x);

    root=(treenode *)malloc(sizeof(treenode));
    root->data=x;
    root->left=NULL;
    root->right=NULL;

    printf("another node y/n? ");
    fflush(stdin);
    opt=getchar();

    while (opt!='n')
    {
        printf ("enter a node value ");
        scanf("%d",&x);
        buildtree(root,x);
        printf("another node y/n? ");
        fflush(stdin);
        opt=getchar();
    }
}

```

```
printf("\nPreorder Traversal \n");
preorder(root);

printf("\nInorder Traversal \n");
inorder(root);

printf("\nPostorder Traversal \n");
postorder(root);

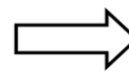
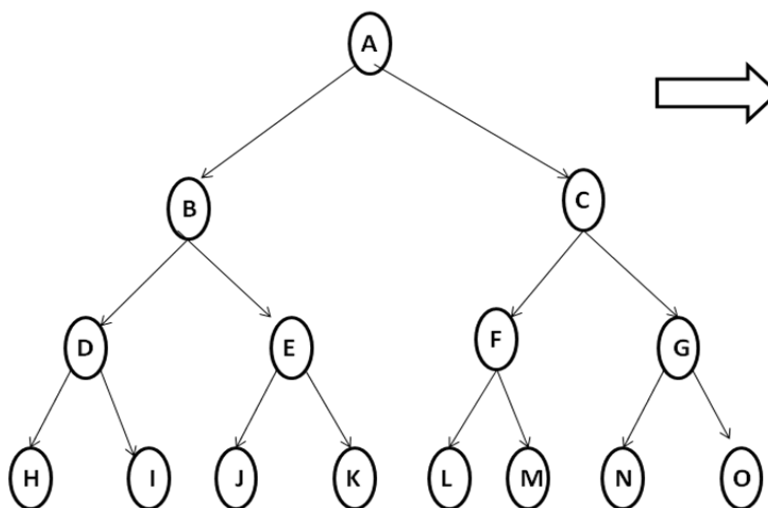
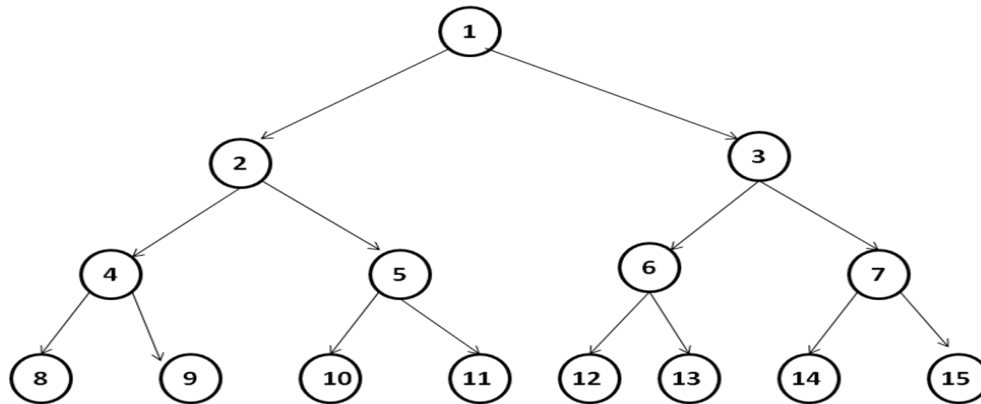
cnt=0;
leafcount(root,&cnt);
printf("\nTotal number of leaf nodes = %d\n",cnt);

ht=height(root);
printf("Height of the binary search tree = %d\n",ht);

getch();
}
```

Representation of Binary Trees using Arrays

Each node is sequentially arranged from top to bottom and from left to right. Let us understand this matter by numbering each node. The numbering will start from root node and then remaining nodes will give ever increasing numbers in level wise direction. The nodes on same level will be numbered from left to right.



ARRAY	
0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H
8	I
9	J
10	K
11	L
12	M
13	N
14	O

Root is stored at location 0. If a node is at location i then its left and right child will be located at positions $2*i+1$ and $2*i+2$.

Example 1) Consider the node B which is at position 1.

Its left child will be at $2*1+1 = 3$ i.e. D

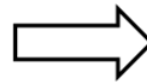
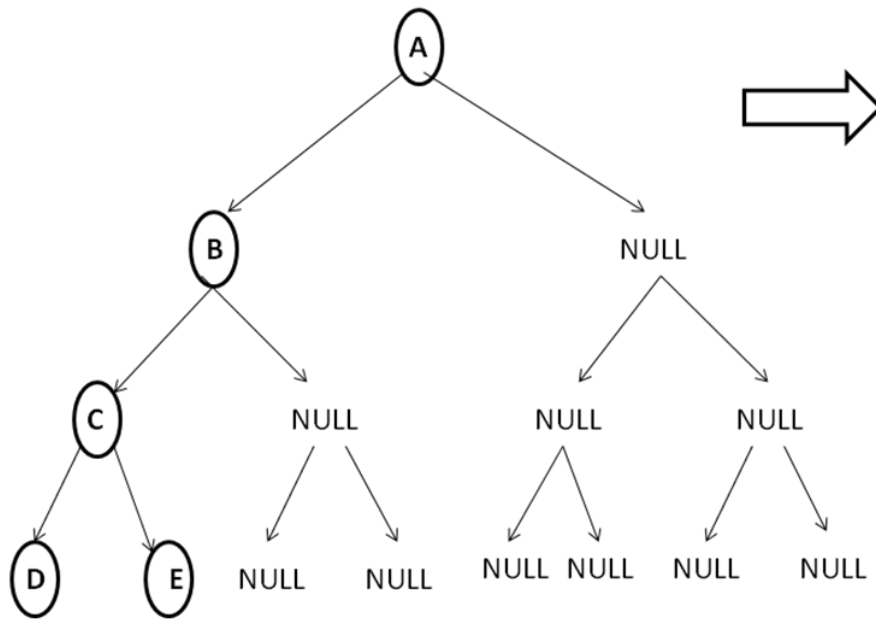
Its right child will be at position $2*1+2 = 4$ i.e. E

Example 2) Consider the node F which is at position 5.

Its left child will be at $2*5+1 = 11$ i.e. L

Its right child will be at position $2*5+2=12$ i.e. M

Consider the following binary tree where there are some NULL nodes.

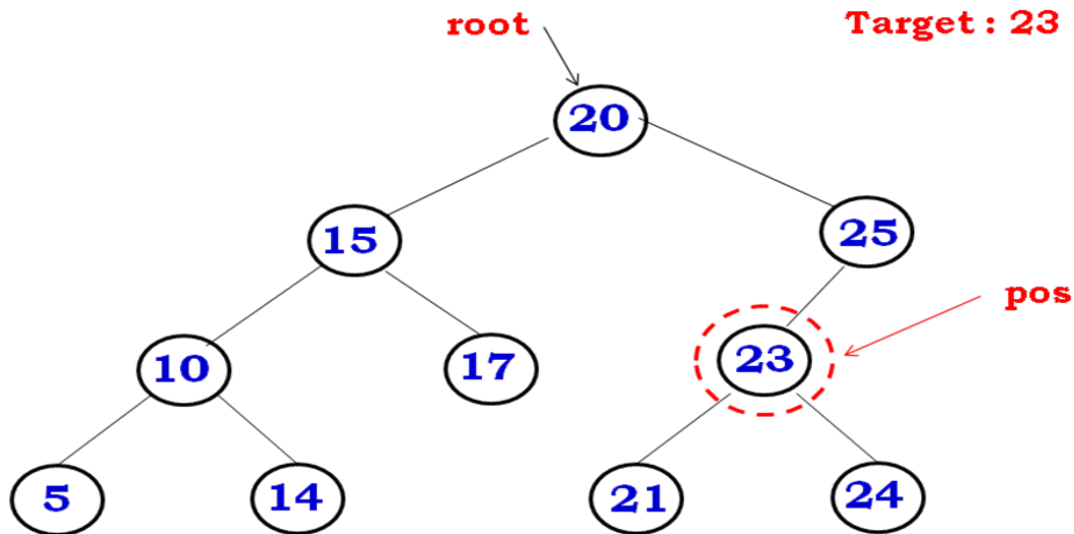


	ARRAY
0	A
1	B
2	
3	C
4	
5	
6	
7	D
8	E
9	
10	
11	
12	
13	
14	

Example 3) Consider the node C which is at position 3.
Its left child will be at $2*3+1 = 7$ i.e. D
Its right child will be at position $2*3+2=8$ i.e E

Tree Search

Tree search is an important operation in Binary Search Tree. To search for the target, we first compare it with the key at the root of the tree. If it is same then searching is finished. If it is not the same, we go to the left subtree or right subtree as appropriate and repeat the search in that subtree.



/-----Finding the position of a node */*

```
treesearch(p,tar,pos)
treenode *p;
int tar;
treenode **pos;
{
    if ( p == NULL )
        *pos = NULL;
    else
        if ( p->data == tar )
            *pos = p;
        else
            if (tar > p->data )
                treesearch(p->right,tar,pos);
            else
                treesearch(p->left,tar,pos);
}
```

```

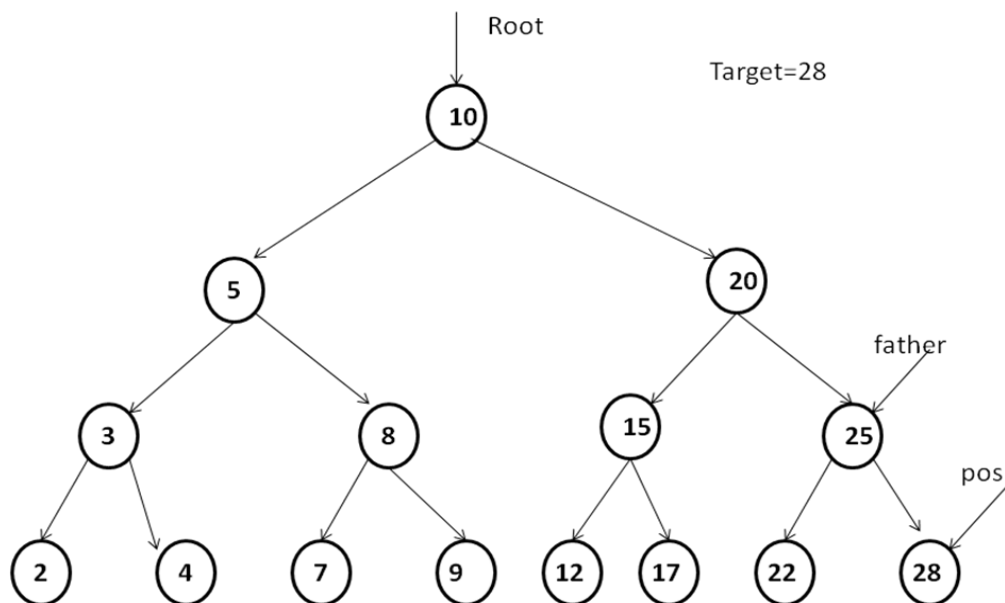
/*-----
    Finding the position of a node
    and the position of its father
-----*/

```

```

treesearch(p,tar,pos,father)
treenode *p;
int tar;
treenode **pos,**father;
{
    if ( p == NULL)
    {
        *father = NULL;
        *pos = NULL;
    }
    else
    if ( p->data == tar)
        *pos = p;
    else
    if (tar > p->data)
    {
        *father = p;
        treesearch(p->right,tar,pos,father);
    }
    else
    {
        *father = p;
        treesearch(p->left,tar,pos,father);
    }
}

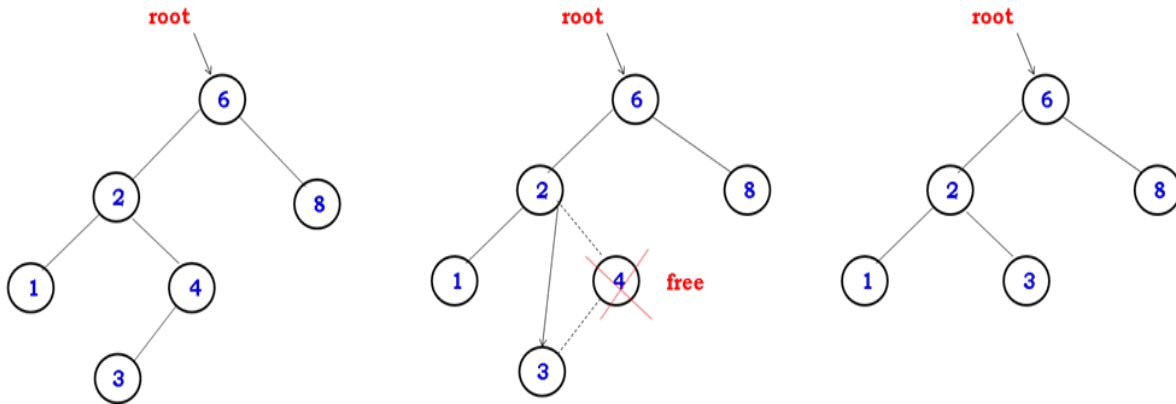
```



Delete Node

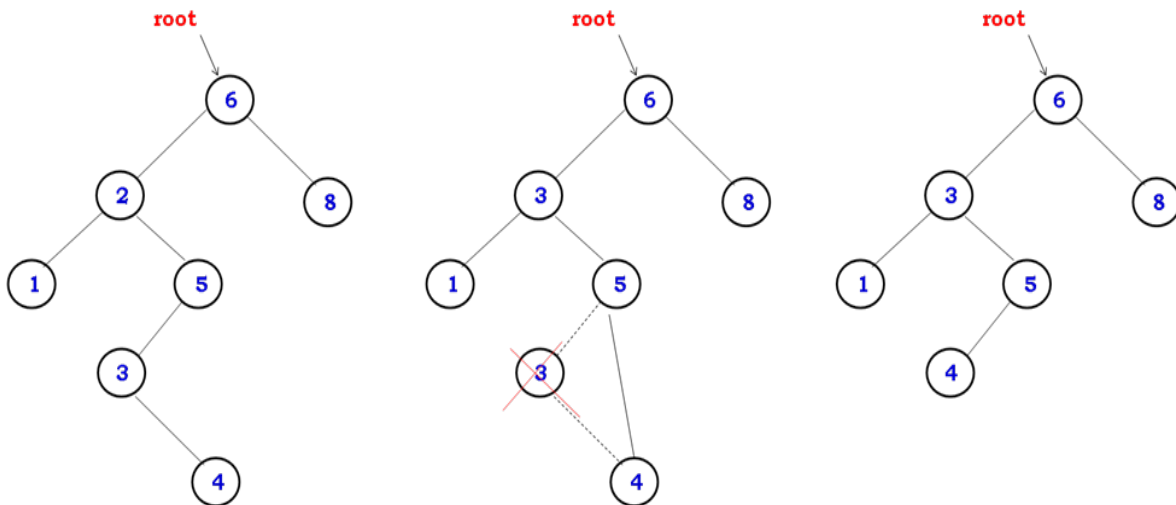
Deleting a node is common with many data structures, the hardest operation is deletion. Once we have found the node to be deleted, we need to consider several possibilities.

If the node is a leaf, it can be deleted immediately. If the node has one child, the node can be deleted after its parent adjusts a pointer to bypass the node.



Deletion of a node (4) with one child

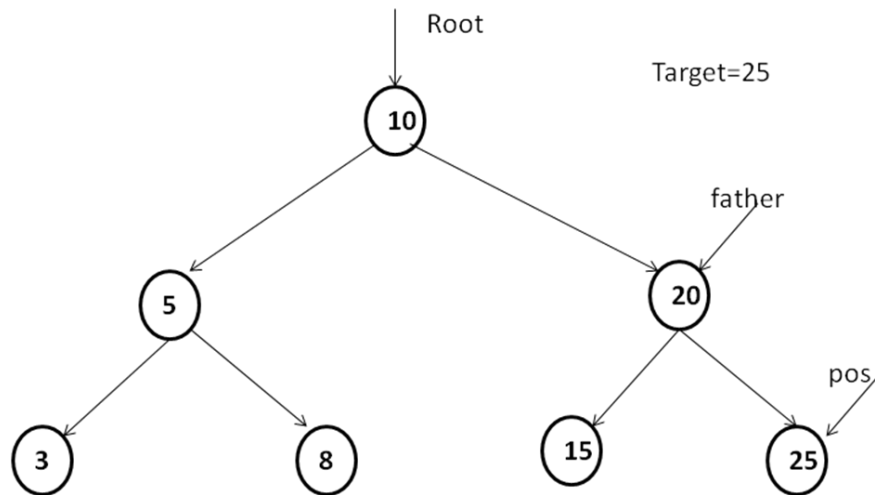
Notice that the deleted node is now unreferenced and can be disposed of only if a pointer to it has been saved. The complicated case deals with a node with two children. The general strategy is to replace the data of this node with the smallest data of the right sub tree and recursively delete that node.



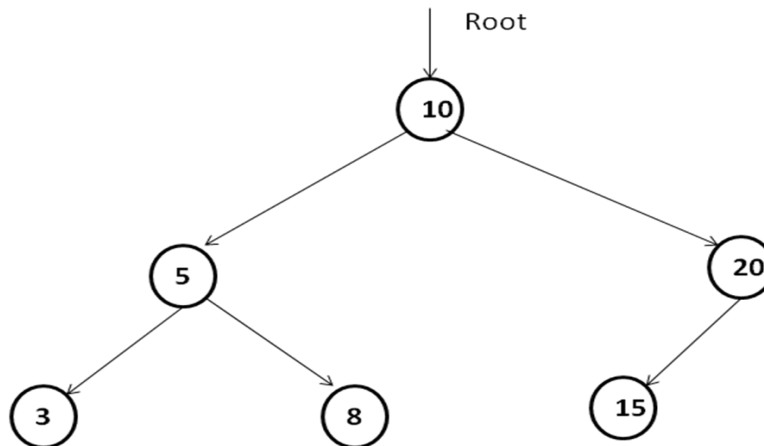
Deletion of node (2) with two children.

Because the smallest node in the right subtree cannot have a left child, the second delete is an easy one. Figure shows an initial tree and the result of a deletion. The node to be deleted is the left child of the root. The key value is 2. It is replaced with the smallest data in its right subtree and then that node is deleted as before.

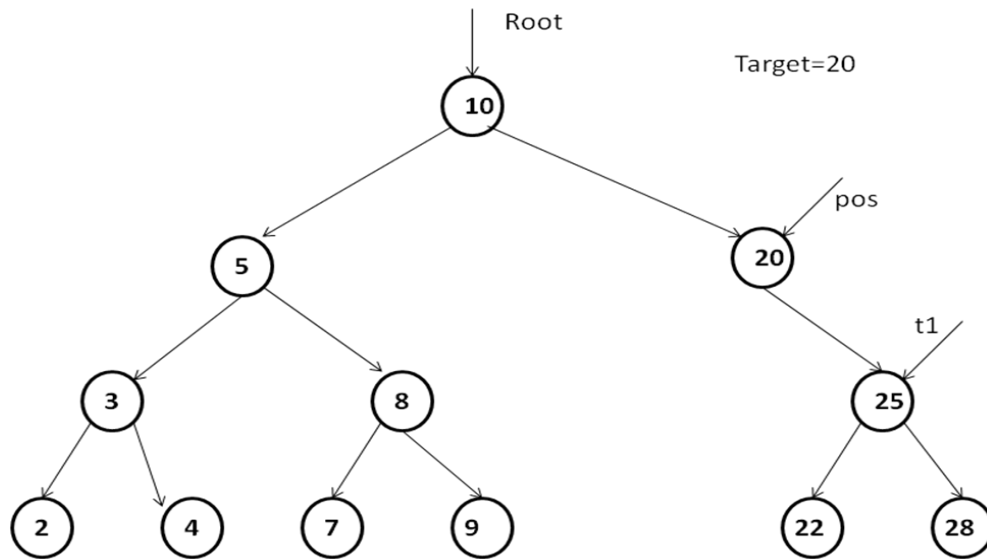
Case 1) Deleting a node having no children i.e deleting a leaf node.



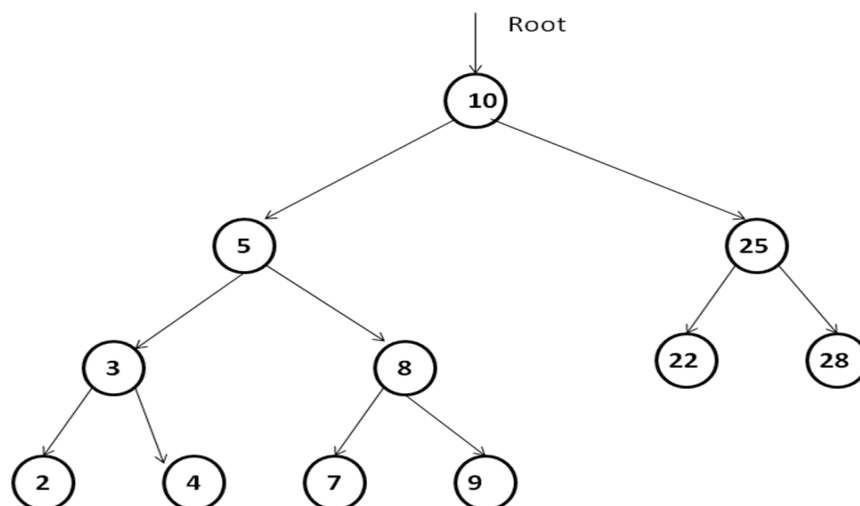
```
if ((pos->left == NULL) && (pos->right == NULL))
{
    if (father->left == pos)
        father->left = NULL;
    else
        father->right = NULL;
    free(pos);
}
```



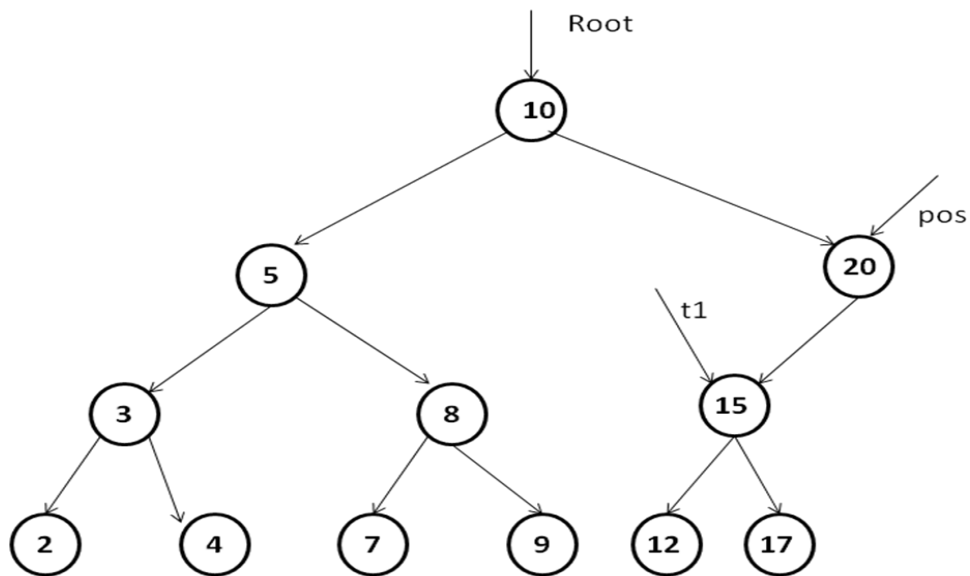
Case 2) Deleting a node having no left children. i.e. the node to be deleted having only right sub tree.



```
if (pos->left==NULL)
{
    t1 = pos->right;
    pos->data=t1->data;
    pos->left=t1->left;
    pos->right=t1->right;
    free(t1);
}
```

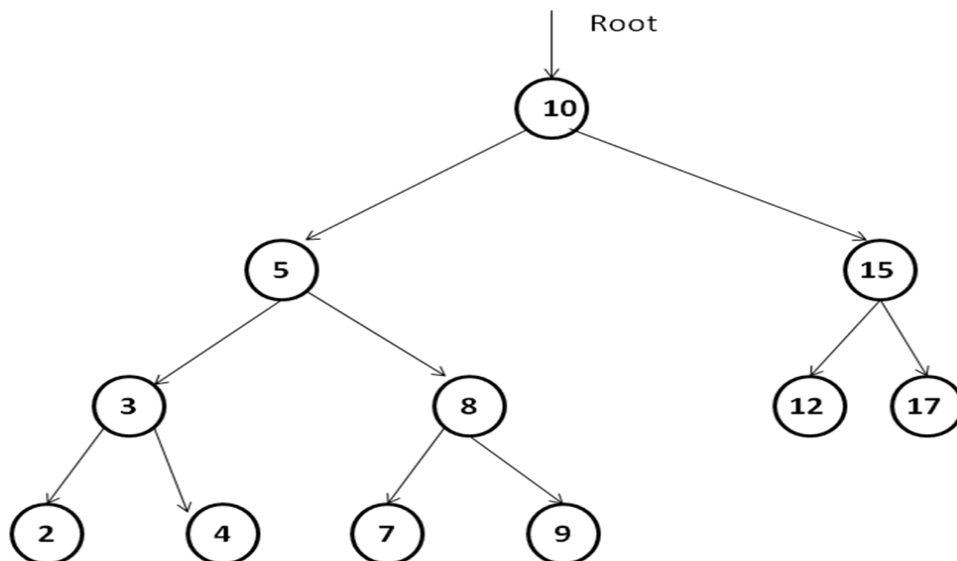


Case 3) Deleting a node having no right children. i.e. the node to be deleted having only left sub tree.

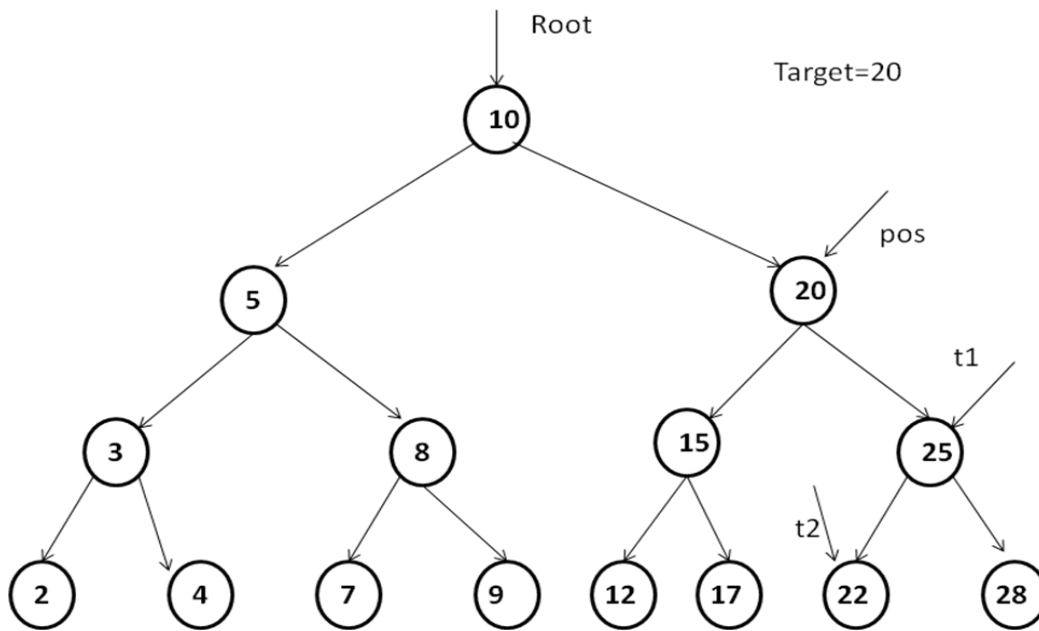


```

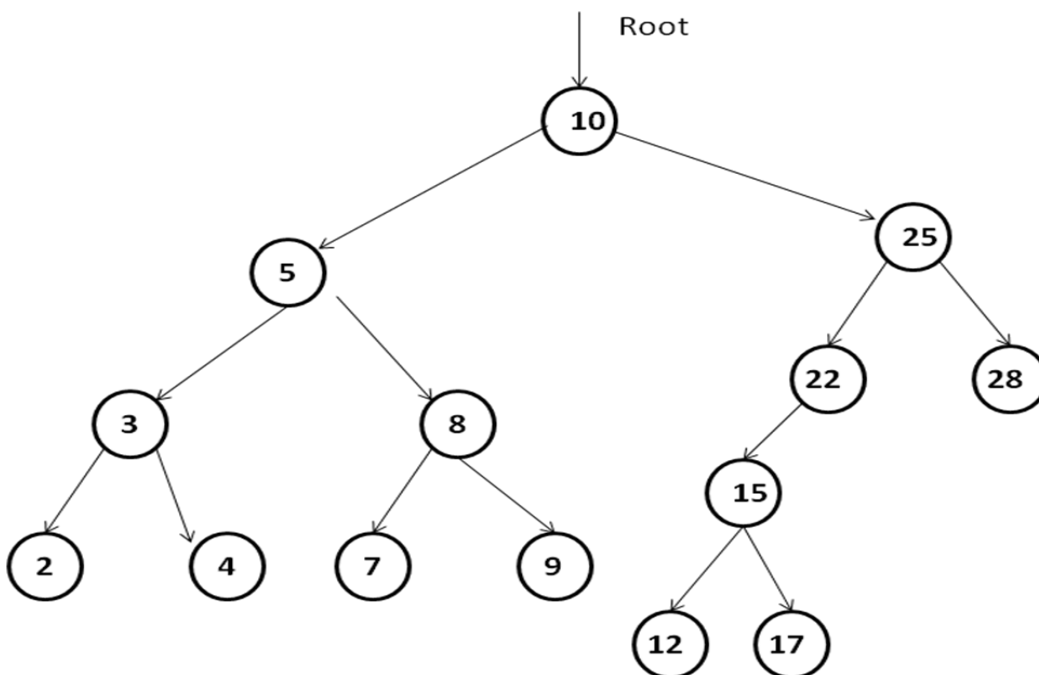
if (pos->right==NULL)
{
    t1 = pos->left;
    pos->data=t1->data;
    pos->left=t1->left;
    pos->right=t1->right;
    free(t1);
}
  
```



Case 4) Deleting a node having both left and right children



```
t1=pos->right;
t2=t1;
while (t2->left!=NULL)
    t2=t2->left;
t2->left=pos->left;
pos->data=t1->data;
pos->left=t1->left;
pos->right=t1->right;
free(t1);
```



/*----- function to delete a node */

```
deletenode(pos, father)
treenode *pos;
treenode *father;
{
    treenode *t1, *t2;
    if ((pos->left == NULL) && (pos->right == NULL))
    {
        if (father->left == pos)
            father->left = NULL;
        else
            father->right = NULL;
        free(pos);
    }
    else
    if (pos->right == NULL)
    {
        t1 = pos->left;
        pos->data = t1->data;
        pos->left = t1->left;
        pos->right = t1->right;
        free(t1);
    }
    else
    if (pos->left == NULL)
    {
        t1 = pos->right;
        pos->data = t1->data;
        pos->left = t1->left;
        pos->right = t1->right;
        free(t1);
    }
    else
    {
        t1 = pos->right;
        t2 = t1;
        while (t2->left != NULL)
            t2 = t2->left;
        t2->left = pos->left;
        pos->data = t1->data;
        pos->left = t1->left;
        pos->right = t1->right;
        free(t1);
    }
}
```

```

/*-----
    Program to Create a Binary Search Tree
    Binary Tree Traversals
    Tree search
    Delete node
-----*/

#include <stdio.h>
#include <conio.h>
#include <malloc.h>

struct node
{
    int data;
    struct node *left;
    struct node *right;
};
typedef struct node treenode;

/*----- Function to create a binary tree */
/*----- adding a node to a binary tree */
void buildtree(p,x)
treenode *p;
int x;
{
    treenode *q;
    if ( x > p->data )
        if (p->right == NULL)
        {
            q=(treenode *)malloc(sizeof(treenode));
            q->data=x;
            q->left=NULL;
            q->right=NULL;
            p->right=q;
        }
        else
            buildtree(p->right,x);
    else
        if ( x < p->data )
            if (p->left == NULL)
            {
                q=(treenode *)malloc(sizeof(treenode));
                q->data=x;
                q->left=NULL;
                q->right=NULL;
                p->left=q;
            }
            else
                buildtree(p->left,x);
    else
        printf("Duplicate node \n");
}

```

```
/*----- Preorder Traversal */
```

```
void preorder(p)
treenode *p;
{
    if (p!=NULL)
    {
        printf("%d ",p->data);
        preorder(p->left);
        preorder(p->right);
    }
}
```

```
/*----- Inorder Traversal */
```

```
void inorder(p)
treenode *p;
{
    if (p!=NULL)
    {
        inorder(p->left);
        printf("%d ",p->data);
        inorder(p->right);
    }
}
```

```
/*----- Postorder Traversal */
```

```
void postorder(p)
treenode *p;
{
    if (p!=NULL)
    {
        postorder(p->left);
        postorder(p->right);
        printf("%d ",p->data);
    }
}
```

```
/* Finding the position of a node and position of its father*/
```

```
void treeearch(p,tar,pos,father)
treenode *p;
int tar;
treenode **pos,**father;
{
    if ( p == NULL)
    {
        *father = NULL;
        *pos = NULL;
    }
    else
        if ( p->data == tar)
            *pos = p;
        else
            if (tar > p->data)
            {
                *father = p;
                treeearch(p->right,tar,pos,father);
            }
}
```

```

    }
else
{
    *father = p;
    treeseach(p->left,tar,pos,father);
}
}

/*-----function to delete node -----*/
void deletenode(pos,father)
treenode *pos;
treenode *father;
{
    treenode *t1,*t2;
    if ((pos->left == NULL)&&(pos->right==NULL))
    {
        if (father->left == pos)
            father->left=NULL;
        else
            father->right=NULL;
        free(pos);
    }
    else
        if (pos->right==NULL)
        {
            t1 = pos->left;
            pos->data=t1->data;
            pos->left=t1->left;
            pos->right=t1->right;
            free(t1);
        }
        else
            if (pos->left==NULL)
            {
                t1 = pos->right;
                pos->data=t1->data;
                pos->left=t1->left;
                pos->right=t1->right;
                free(t1);
            }
        else
        {
            t1=pos->right;
            t2=t1;
            while (t2->left!=NULL)
                t2=t2->left;
            t2->left=pos->left;
            pos->data=t1->data;
            pos->left=t1->left;
            pos->right=t1->right;
            free(t1);
        }
    }
}

```

```

void main()
{
    treenode *root,*pos,*father;
    char opt;
    int x,target;

    clrscr();

    printf ("enter the root value ");
    scanf ("%d",&x);

    root=(treenode *)malloc(sizeof(treenode));
    root->data=x;
    root->left=NULL;
    root->right=NULL;

    printf("another node y/n? ");
    fflush(stdin);
    opt=getchar();

    while (opt!='n')
    {
        printf ("enter a node value ");
        scanf ("%d",&x);

        buildtree(root,x);

        printf("another node y/n? ");
        fflush(stdin);
        opt=getchar();
    }
    printf("\nPreorder Traversal \n");
    preorder(root);

    printf("\nInorder Traversal \n");
    inorder(root);

    printf("\nPostorder Traversal \n");
    postorder(root);

    printf("\nEnter target node data ");
    scanf ("%d",&target);

    pos = NULL;
    father = NULL;

    treeseach(root,target,&pos,&father);

    printf("position data  %d\n",pos->data);
    printf("fathers data  %d\n",father->data);
}

```

```

if ( pos == NULL )
    printf("the target node not found \n");
else
{
    deletenode(pos, father);

    printf("\nTree traversals after deleting \n");

    printf("\nPreorder Traversal \n");
    preorder(root);

    printf("\nInorder Traversal \n");
    inorder(root);

    printf("\nPostorder Traversal \n");
    postorder(root);
}
getch();
}

```

Important Questions

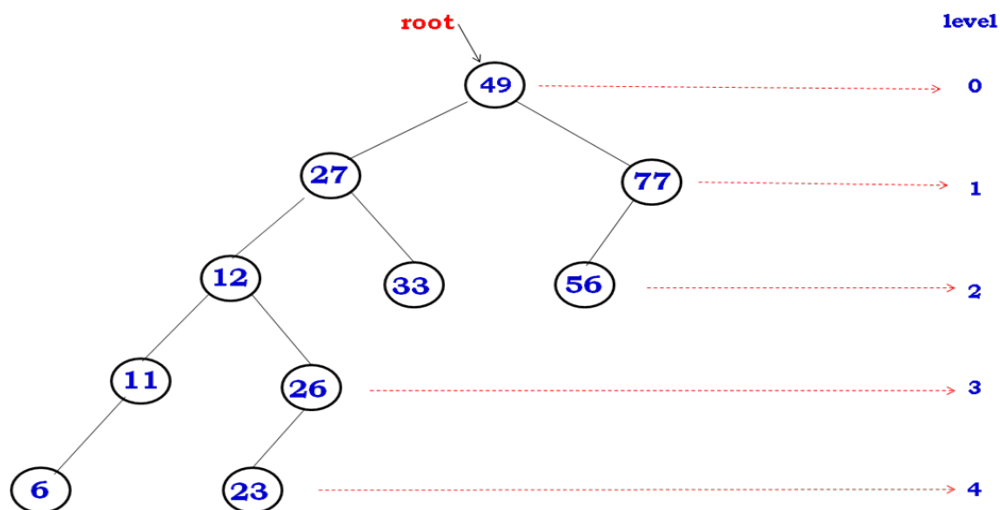
1. What is a binary tree? Explain about representation of binary trees using arrays.
2. Explain about representation of binary trees using linked lists.
3. Explain operations on binary trees.

insert
search
inorder
preorder
postorder
delete

4. Write a program to create a binary tree.
5. Write a program to delete a node from a binary tree.
6. Explain in detail the preorder, inorder and postorder traversals of a binary tree.
7. What are different tree traversals? Explain with examples.
8. Write a program to traverse a binary tree in all the three orders.
9. Build a binary tree from preorder and inorder traversals.
Preorder: ABDCE
inorder: DBACE
10. Build a binary tree from preorder and inorder traversals.
Preorder: ABCDEFGHI
inorder: BCAEDGHI
11. Build a binary tree from postorder and inorder traversals.
Postorder: HIDEBFGCA
Inorder: HDIBEAFCG
12. Define a full binary tree.
13. What is a binary tree? Construct a binary tree given the pre-order traversal and inorder traversals as follows:
Pre-Order Traversal: G B Q A C K F P D E R H
In-Order Traversal: Q B K C F A G P E D H R
14. Show that the maximum number of nodes in a binary tree of height H is $2^{H+1} - 1$.
15. Define path in a tree.

16. Define binary search tree. Show how to insert and delete an element from binary search tree.
17. Write algorithm to insert and delete an element from binary search tree.
18. Explain the properties of a binary search tree in detail.
19. Develop a binary search tree resulting after inserting the following integer keys 49, 27, 12, 11, 33, 77, 26, 56, 23, 6.
 - i) Check whether the tree is almost complete or not?
 - ii) Determine the height of the tree
 - iii) Write post order and preorder traversals.

Solution



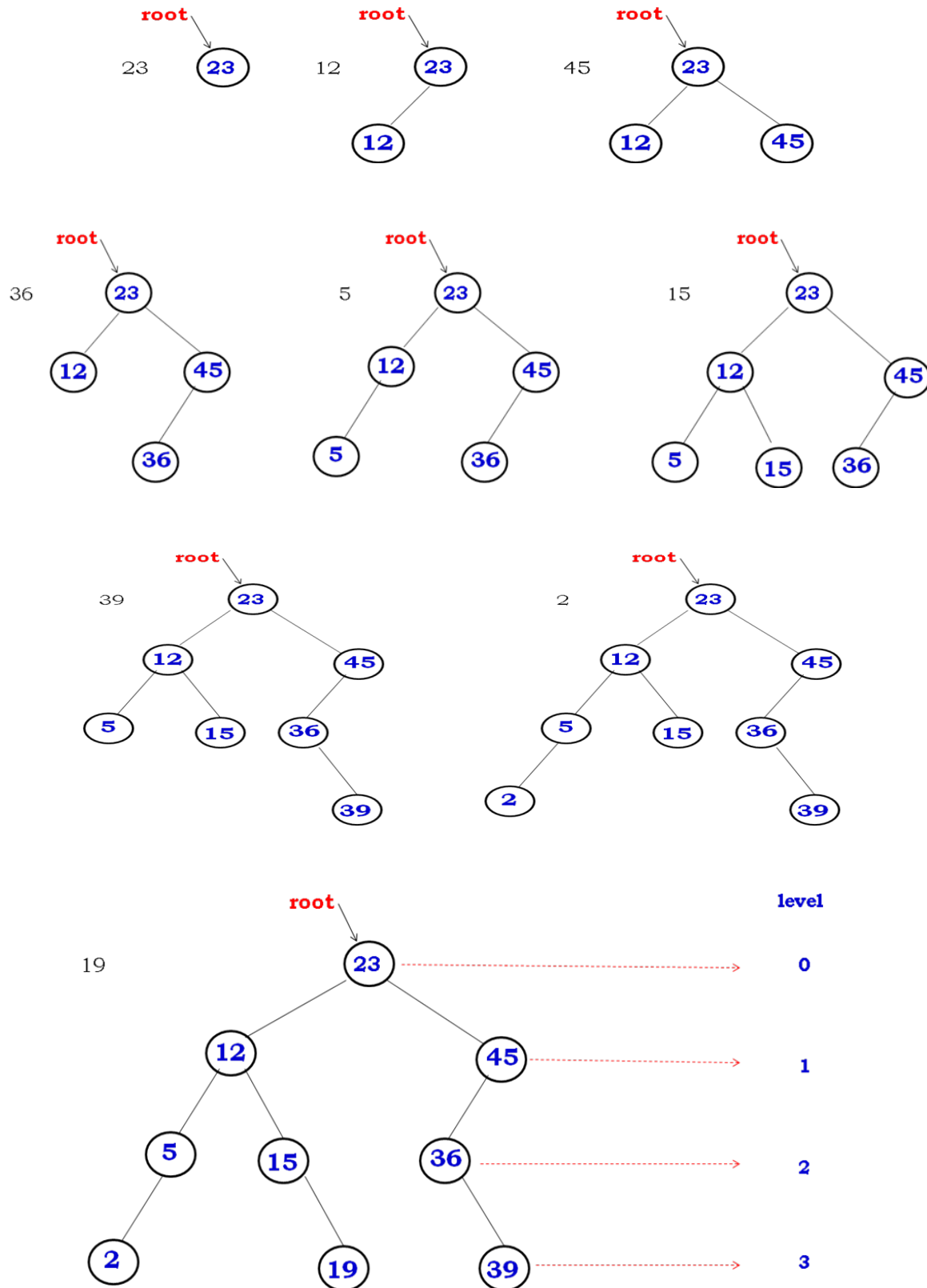
- i) The above binary search tree is not a complete binary tree
 - ii) Height is 4
 - iii) post order 6, 11, 23, 26, 12, 33, 27, 56, 77, 49
 - iv) preorder 49, 27, 12, 11, 6, 26, 23, 33, 77, 56
20. How to represent binary tree using arrays and linked list?
 21. Write in-order, pre-order and post-order traversal of a binary tree.
 22. What are the features of a threaded binary tree?
 23. A binary tree has seven nodes. The Preorder and Postorder traversal of the tree are given below. Can you draw the tree? Justify.
 Preorder : GFDABEC
 Postorder : ABDCEFG

24. Create binary search tree for the following elements

23, 12, 45, 36, 5, 15, 39, 2, 19

Discuss about the height of the above binary search tree.

solution



A Binary Search tree of height /depth 3

Threaded Binary Tree

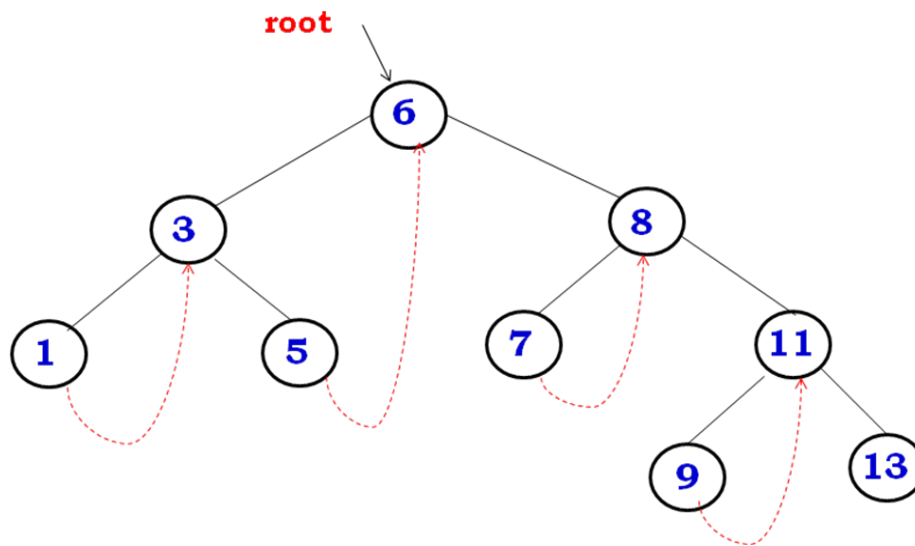
The idea of threaded binary trees is to make inorder traversal faster and do it without stack and without recursion. A binary tree is made threaded by making all right child pointers that would normally be NULL point to the inorder successor of the node (if it exists).

There are two types of threaded binary trees.

Single Threaded: Where a NULL right pointers is made to point to the inorder successor (if successor exists)

Double Threaded: Where both left and right NULL pointers are made to point to inorder predecessor and inorder successor respectively. The predecessor threads are useful for reverse inorder traversal and postorder traversal.

The threads are also useful for fast accessing ancestors of a node. Following diagram shows an example Single Threaded Binary Tree. The dotted lines represent threads.



Double Threaded Binary Search Tree

Double Threaded Binary Search Tree is a binary search tree in which the nodes are not every left NULL pointer points to its inorder predecessor and the right NULL pointer points to the inorder successor.

The threads are also useful for fast accessing the ancestors of a node.

Double Threaded Binary Search Tree is one of the most used types of Advanced data structures used in many real-time applications like places where there are recent insertion and traversal of all elements of the search tree.

